

MTD 367 iOS Application Development

Introduction

- Fadzuli
- Masters in Entertainment Technology, Carnegie Mellon Univeristy
- 14 years in the gaming industry
 -  Disney
 -  King
 -  Mighty Bear Games

Course Outline

- **Lesson #1 - Building Apps That Matter & Swift (Today)**
- Lesson #2 - Dealing With Swift Types & SwiftUI (11/8/2025)
- Lesson #3 - Doubling Down On SwiftUI (18/8/2025)
- Lesson #4 - All To-Do With SwiftData (1/9/2025)
- Lesson #5 - Streaming Multimedia Swiftly (8/9/2025)
- Lesson #6 - Putting The Map On Your App (15/9/2025)

Virtual Labs

-  Blackjack
 - Virtual Lab #1 (9/8/2025)
 - Virtual Lab #2 (16/8/2025)
 - Virtual Lab #3 (23/8/2025)
-  To-Do List App
 - Virtual Lab #4 (6/8/2025)
 - Virtual Lab #5 (13/9/2025)
 - Virtual Lab #6 (20/9/2025)

✓ Today's Agenda

- ☐ Course Objectives & Expectations
- ☐ What Is Swift & Why It Matters?
- ☐ Basic Swift Language Features

Course Objectives & Expectations

- Expect basic programming knowledge
- No Swift experience required
- Git/GitHub not required but helpful

Grading Breakdown

Assessment	Weight	Due Date
TMA 1	15%	25/8/2025
TMA 2	15%	22/9/2025
ECA	70%	14/10/2025

Online Learning

- Use emoji reactions on Zoom   
 - Helps me know if I should explain topics further
- Ask questions in the Zoom chat anytime
- Participate in activities and polls
- Need help after class?
 - Email me: muhammadfadzuli001@suss.edu.sg
 - I'll reply within 24 hours

How To Ask For Help

Good questions are:

- Clear and specific
- Explain what you've tried
- Include code snippets
 - Share using GitHub gist
- Describe:
 - What you expected
 - What actually happened

✓ Good Example #1

“ “I’m trying to load an image using UIImage(named:) but it keeps returning nil. Here’s the code I’m using.

```
let image = UIImage(named: "background.png")
```

The image is in my Xcode project under Assets.xcassets. I expected it to display on screen, but nothing shows up. What could be wrong?”

”

✓ Good Example #2

“ “I’m animating a falling shape in SwiftUI using an explicit animation:

```
withAnimation(.linear(duration: 1.0)) {  
    yOffset = -300  
}
```

But the shape only moves partway down the screen instead of fully reaching the target offset. I expected it to end at -300, but it stops around -150. What could be causing this?” ”

✗ Bad Examples

“ My code doesn't work. Help. ”



“ Can someone fix my whole project? ”



My Stance On AI

- **Use it!**
- Understand what AI is suggesting
 - But adapt it into code you can read and understand
-  AI “cheats”
 - Be Critical of AI
 - Sometimes AI gives incorrect or incomplete answers
 - Recognizing bad AI output is a skill

🧠 This Course Is NOT...

❌ A Matrix-style download of all Swift knowledge.



✓ Instead...

- We'll work through **hands-on examples**
- Learning Swift features only **when they're relevant**
- My Teaching Style:
 - **Incrementalism:** Learn bit by bit
 - **Continuous Refactoring:** Improve code gradually
 - **Fail Fast:** Test quickly and learn from mistakes

macOS Requirement

- Swift is open source → can run on Linux, Windows.
- But:
 - **Xcode only runs on macOS.**
 - You need macOS to build iOS apps
- Our course focuses on iOS → you'll need a Mac

What Is Swift & Why It Matters?

- Modern programming language from Apple
- Released in 2014
- Aims to:
 - Be safer than Objective-C
 - Be faster than interpreted languages
 - Replace C and C++ for Apple platforms

A Brief History

- Before Swift:
 - Apps were written in **Objective-C**
 - Verbose syntax and manual memory management
- Apple wants:
 - Safer code
 - Modern syntax
 - Cross-platform potential

Why Learn Swift?

- Mobile devices are everywhere!
- iOS users generally spend more on apps and in-app purchases
- High demand for skilled mobile developers
-  Prepared for a possible future: AR & VR
 - Tech CEOs see AR and VR as the next big platform
 - **visionOS** → Swift works for building apps for Vision Pro

Build Local Solutions

- We shouldn't rely only on big tech or foreign companies
- **Local developers understand local needs best**

Basic Swift Language Features

Swift Concepts	Details
 Variables	<code>let</code> → constant <code>var</code> → variable
 Data types	<code>String</code> , <code>Int</code> , <code>Double</code> , <code>Bool</code> , <code>Optional</code>
 Control Statements	<code>if</code> / <code>else</code> , <code>for</code> , <code>guard</code>
 Functions	How to define and call functions
 Data structures	<code>Array</code>

Xcode

We will be looking at the above features and working on Hands-On #1 (Functions).





Hands-On #2

Build a simple Blackjack game in Playground:

- Deal cards
- Calculate hand totals
- Determine hand value
 - `Blackjack == 21`
 - `Busted > 21`
- Add a second player
- Determined who won

✓ Summary

- ✓ Course Objectives & Expectations
- ✓ What Swift Is?
- ✓ Why They Matter?
- ✓ Basic Swift Language Features
 - Variables: `let`, `var`
 - Data types: `String`, `Int`, `Double`, `Bool`, `Optional`
 - Data structure: `Array`
 - Control Statements: `if` / `else`, `for`, `guard`

Deep Dive on Swift if you have 7 hours

[Vandad's Full Swift Programming Course for Beginners](#)



Virtual Lab #1 (9/8/2025)

Blackjack doesn't stop after the first two cards.

- Players can:
 - **Hit** → take another card
 - **Stand** → keep their current total
- Simulate choices with random decisions.
- The next hand starts once a winner has been determined for a single hand.

? Questions?

I'd love to hear how the class went or how I can improve.

Feel free to email me at:

muhammadfadzuli001@suss.edu.sg

 All course materials are available at:

www.github.com/iluzdaf/mtd367

Appendix

"Hello World"

```
import Foundation
```

```
var greeting = "Hello World"
```

```
print(greeting) // In a regular Xcode project, you'd need to print it:
```

```
greeting // In a Playground, you can simply type the variable name:
```

```
// Playgrounds automatically run your code each time you make changes,  
// so you can see your results instantly.
```



let vs **var**

- **let**
 - Constant value
 - Cannot change after it's set
- **var**
 - Variable value
 - Can change later

Prefer `let` by default

- If you don't need to change it, make it a constant
 - Prevents accidental changes
 - Makes code safer and easier to understand
- Constants Improve Performance
 - The compiler can optimize code better if it knows values won't change

```
let name = "Fadzuli"  
name = "Ali" //X
```

```
var age = 43  
age = 44  
age += 1
```

// Think of let as something fixed, and var as something flexible.

```
let city = "Singapore"  
var temperature = 28  
temperature = 30  
// because weather changes!
```



String

```
// Text or words  
let name: String = "Fadzuli"  
let myString = ""  
let count = myString.count  
myString.uppercased()  
myString.lowercased()  
myString.isEmpty
```




Int

```
// Whole numbers
var age: Int = 43
let sum = 2 + 3
let product = age * 2
var score = 0
score += 1
if (age > 42)
{
    "Hello grandpa!"
}
```



Double

```
// Decimal numbers  
let height: Double = 1.75  
height.rounded()  
height.rounded(.down)  
let heightMessage = String(format: "%.2f", height)
```



Bool

```
var isStudent: Bool = false // Yes/No
if isStudent {
    "Welcome, student!"
}
isStudent.toggle()
```



Optional

```
// Optionals let us handle missing data safely
var nickname: String? = nil // I may or may not have a nickname

// In order to use nickname,
// You need to unwrap the nickname safely
// Safely means checking if your optional has a value before using it
// If you try to use it without checking, your program might crash
if let shortName = nickname {
    " My friends call me \$(shortName)."
}
else {
    " I have no friends."
}
```

Functions

```
// When you define a Swift function,  
// you can give your parameters external names to make your code read like a sentence.  
// Easier to read  
// Self-documenting  
// Less error-prone  
func introduce(withName name: String, andAge age: Int, andHeight height: Double, andNickname nickname: String?) -> String {  
    ...  
}  
  
// You can also do away with external names  
func max(_ value1: Int, _ value2: Int) -> Int {  
    ...  
}
```



if/else

```
func max(_ value1: Int, _ value2: Int) -> Int {  
  if value1 > value2 {  
    return value1  
  } else {  
    return value2  
  }  
}
```

// or in tenary form

```
func max(_ value1: Int, _ value2: Int) -> Int {  
  return value1 > value2 ? value1 : value2  
}
```



for

```
func calculateHandValue(_ cards: [Int]) -> Int {  
    var total = 0  
    for card in cards {  
        total += card  
    }  
    return total  
}
```



guard

```
func introduce(  
  withName name: String, andAge age: Int,  
  andHeight height: Double,  
  andNickname nickname: String?) -> String {  
  var intro = "Hi! My name is \$(name). I am \$(age) years old. My height is \$(height) meters."  
  
  guard let shortName = nickname else {  
    intro += " I have no friends"  
  
    return intro  
  }  
  
  intro += " My friends call me \$(shortName)."  
  
  return intro  
}
```




Array

```
// An Array is a Collection of values, all kept together in one variable.  
// You can store many items of the same type-like a row of boxes side by side  
var fruits: [String] = ["Apple", "Banana", "Durian"]
```

```
// Arrays are zero-indexed → first item is at index 0.  
fruits[0]  
fruits[2]
```

```
// Changing an element  
fruits[1] = "Mango"  
fruits
```

```
// Adding  
fruits.append("Pineapple")
```

```
// Removing  
fruits.remove(at: 2)  
fruits
```

```
// Creating empty array
var cards = [Int]()
cards.append(11)
cards.append(10)

// Looping
func calculateHandValue(_ cards: [Int]) -> Int {
    var total = 0

    for card in cards {
        total += card
    }

    return total
}

let handValue = calculateHandValue(cards)
if handValue == 21 {
    "Blackjack!"
}
else if handValue > 21 {
    "You busted!"
}
else {
    "Your hand value is \(handValue)."
}
```